

```

// Headless verification of Demo 26's ray-space math
const Zv = 150, Zh = 250, D = Zh - Zv;
const rayAt = (a,z)=>{const l=(z-Zv)/D; return [a[1]*l, a[0]*(1-l), z];};
const foot1=(a)=>[0,a[0],Zv], foot2=(a)=>[a[1],0,Zh];
const dir=(a)=>[a[1],-a[0],D];
const dist=(a,b)=>{
  const p=[0,a[0]-b[0],0], u=dir(a), v=dir(b);
  const c=[u[1]*v[2]-u[2]*v[1],u[2]*v[0]-u[0]*v[2],u[0]*v[1]-u[1]*v[0]];
  const n=Math.hypot(...c);
  return n<1e-9?0:Math.abs(p[0]*c[0]+p[1]*c[1]+p[2]*c[2])/n;};

// 1) ray really passes through both feet
let w=0;
for(let i=0;i<10000;i++){
  const a=[(Math.random()-0.5)*340,(Math.random()-0.5)*340];
  const r1=rayAt(a,Zv), r2=rayAt(a,Zh);
  w=Math.max(w, Math.hypot(r1[0]-0,r1[1]-a[0]), Math.hypot(r2[0]-a[1],r2[1]-0));
}
console.log("feet incidence worst-case      :", w.toExponential(2));

// 2) chart consistency (D5): screen point = (-1.5 t, 2.5 s)
w=0;
for(let i=0;i<10000;i++){
  const a=[(Math.random()-0.5)*340,(Math.random()-0.5)*340];
  const S=rayAt(a,0);
  w=Math.max(w, Math.abs(S[0]+1.5*a[1]), Math.abs(S[1]-2.5*a[0]));
}
console.log("screen chart (-1.5t, 2.5s) err :", w.toExponential(2));

// 3) C2 dichotomy: distance 0 iff shared foot
let bad=0, minSkew=1e9, maxPar=0;
for(let i=0;i<20000;i++){
  const a=[(Math.random()-0.5)*340,(Math.random()-0.5)*340];
  const b=[(Math.random()-0.5)*340,(Math.random()-0.5)*340];
  const d=dist(a,b);
  const shared = a[0]===b[0]||a[1]===b[1];
  if(!shared && Math.abs(a[0]-b[0])>1 && Math.abs(a[1]-b[1])>1)
    minSkew=Math.min(minSkew,d);
  // force shared-foot pairs too
  const c=[a[0], b[1]]; // shares s with a
  maxPar=Math.max(maxPar, dist(a,c));
}
console.log("min gap, generic pairs (>0)      :", minSkew.toFixed(4));
console.log("max gap, 1-parallel pairs (=0)    :", maxPar.toExponential(2));

```

```

// 4) S-pencil planarity (C3): rays {s=s0, t varying} all lie in span(F1,l2)
const s0=55, F1=[0,s0,Zv];
// plane through F1 containing l2 (dir (1,0,0)) and F1->(0,0,Zh):
// normal = (1,0,0) x (0-0, 0-s0, Zh-Zv) = (0*D-0*(-s0), 0*0-1*D, 1*(-s0)-0)
const nrm=[0,-D,-s0];
w=0;
for(let t=-170;t<=170;t+=10){
  for(const z of [0, 100, 300, 440]){
    const p=rayAt([s0,t],z);
    const dev=Math.abs(nrm[0]*(p[0]-F1[0])+nrm[1]*(p[1]-F1[1])+nrm[2]*(p[2]-F1[2])
      /Math.hypot(...nrm);
    w=Math.max(w,dev);
  }
}
console.log("S-pencil planarity worst-case  :", w.toExponential(2));

```